

ESCOLINHA DE IA · MÓDULO 05 · REVISÃO PARA O LABORATÓRIO 1

REVISÃO 1 DE 2 · AULAS 5.1, 5.2 E 5.3

Do dataset cru até o Pipeline

EDA, X e y, treino e teste, vazamento de dados, pré-processamento, ColumnTransformer e Pipeline — com o Titanic do começo ao fim.

2 HORAS

DATASET: TITANIC

PRÓXIMO: REVISÃO 2 · FEATURES, VALIDAÇÃO E MÉTRICAS

Ao final desta aula, você vai conseguir

- 1 Explicar o Titanic como um problema de classificação supervisionada, com X e y definidos.
- 2 Fazer uma EDA básica e transformar cada resultado em uma decisão de ML.
- 3 Separar treino e teste e reconhecer quatro formas de vazamento de dados.
- 4 Justificar imputação, escalonamento e One-Hot para cada coluna do Titanic.
- 5 Montar um ColumnTransformer e um Pipeline, explicando cada linha.

Onde estamos no Módulo 5

Paramos na 5.5. Estas duas revisões não trazem conteúdo novo: organizam o que já vimos para o Laboratório 1.

5.1

Pipeline completo, do dado cru à métrica

HOJE

5.2

Treino, validação, teste e vazamento

HOJE

5.3

Pipeline e ColumnTransformer

HOJE

5.4

Feature engineering

REVISÃO 2

5.5

Seleção de features

REVISÃO 2

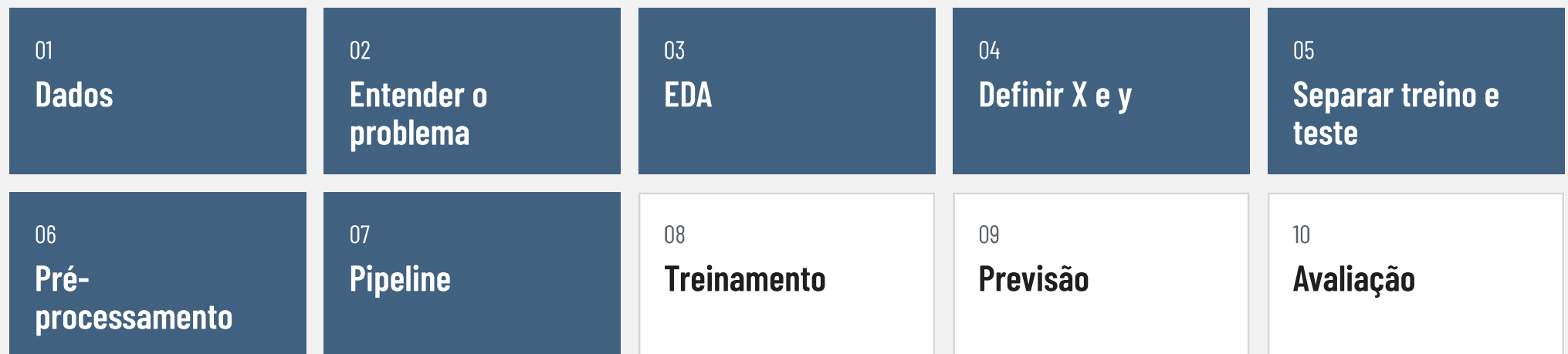
5.6

Laboratório 1: Titanic de ponta a ponta

ENTREGA

O mapa geral de um projeto de ML

Este é praticamente o caminho que o seu notebook do laboratório vai percorrer. Hoje cobrimos as etapas em azul.



Leia da esquerda para a direita, linha por linha. Treinamento, previsão e avaliação aparecem hoje no Pipeline e são aprofundados na Revisão 2.

03 · REVISÃO TEÓRICA · PARTE 1

O problema do Titanic

Aprendizado supervisionado, classificação, linhas, colunas, alvo e features.

Aprendizado supervisionado

CONCEITO

O modelo aprende a partir de exemplos em que a resposta certa já é conhecida.

INTUIÇÃO

Como estudar com uma lista de exercícios resolvidos: você vê as perguntas e os gabaritos, e depois tenta resolver questões novas.

NO TITANIC

Temos 891 passageiros e sabemos quem sobreviveu. O modelo aprende esse padrão para prever passageiros que ele não viu.

Características do passageiro



Modelo



Sobreviveu? 0 ou 1

Por que o Titanic é classificação

A resposta que queremos prever é uma categoria, não um número contínuo.

CLASSIFICAÇÃO

Prevê uma classe. Ex.: sobreviveu ou não sobreviveu. Espécie da flor no Iris (Aula 5.1).

REGRESSÃO

Prevê um número. Ex.: preço de uma casa no California Housing (Aulas 5.2 e 5.5).

0

não sobreviveu

1

sobreviveu

Duas classes possíveis: por isso chamamos de classificação binária.

Cada linha é um passageiro, cada coluna é uma característica

Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.25	S
1	1	female	38.0	1	0	71.28	C
1	3	female	26.0	0	0	7.93	S
1	1	female	35.0	1	0	53.10	S
0	3	male	NaN	0	0	8.46	Q

Linha = um exemplo, um passageiro.

Coluna = uma informação sobre ele.

Survived = o que queremos prever.

Features (X) e variável alvo (y)

X · as features, a entrada

Pclass	classe do bilhete: 1, 2 ou 3
Sex	sexo: male ou female
Age	idade em anos
SibSp	irmãos e cônjuges a bordo
Parch	pais e filhos a bordo
Fare	valor pago pela passagem
Embarked	porto: S, C ou Q

y · o alvo, a saída

Survived

É a resposta que o modelo tenta acertar. Nunca entra como pista.

03 · REVISÃO TEÓRICA · PARTE 2

EDA: conhecer os dados antes do modelo

Seis comandos, e cada um responde a uma pergunta.

Quatro perguntas para cada comando

Um comando de EDA só vale a pena se o resultado virar uma decisão. Nos próximos slides, todo comando é lido nesta ordem.

1**Pergunta**

O que estamos perguntando ao dataset?

2**Retorno**

O que o comando devolve?

3**Observar**

Onde olhar no resultado?

4**Decisão**

Que escolha de ML nasce dali?

EDA 1 e 2: head() e shape

```
df.head()
```

PERGUNTA	Como os dados se parecem?
RETORNO	As 5 primeiras linhas.
OBSERVAR	Nomes das colunas, textos, números, NaN já visíveis.
DECISÃO	Sex e Embarked são texto: vão precisar de codificação.

```
df.shape # (891, 12)
```

PERGUNTA	Qual o tamanho do dataset?
RETORNO	(linhas, colunas). Sem parênteses: é atributo.
OBSERVAR	891 passageiros é um dataset pequeno.
DECISÃO	Apagar linhas custa caro. Um único split pode variar muito: validação cruzada ajuda.

EDA 3 e 4: info() e describe()

df.info()

PERGUNTA	Qual o tipo de cada coluna e quantos valores existem?
RETORNO	Dtype (int64, float64, object) e contagem de não nulos.
OBSERVAR	Colunas object são texto. Age tem 714 de 891 valores.
DECISÃO	Primeira separação entre colunas numéricas e categóricas.

df.describe()

PERGUNTA	Como os números se distribuem?
RETORNO	count, mean, std, min, quartis e max das colunas numéricas.
OBSERVAR	Fare: média 32, mediana 14, máximo 512. Valores extremos puxam a média.
DECISÃO	Escalas muito diferentes pedem escalonamento. Outliers influenciam a escolha entre média e mediana.

EDA 5: isnull().sum()

```
df.isnull().sum()
```

PERGUNTA	Quais colunas possuem dados ausentes?
RETORNO	isnull() marca True onde há NaN; sum() conta os True por coluna.
OBSERVAR	Quanto falta em cada coluna, em proporção ao total de 891.
DECISÃO	Imputar Age e Embarked. Retirar Cabin, que está quase toda vazia.



Valores do train.csv do Titanic (891 linhas).

EDA 6: value_counts() do alvo

```
df["Survived"].value_counts()
```

PERGUNTA	As classes estão equilibradas?
RETORNO	Quantas vezes cada valor aparece.
OBSERVAR	Há mais não sobreviventes do que sobreviventes.
DECISÃO	Usar stratify=y no split e não confiar só na acurácia.

0 · 549

61,6%

1 · 342

38,4%

Um "modelo" que sempre responde 0 já acerta **61,6%**. Esse é o piso: qualquer modelo precisa ficar acima dele para ter valor.

Tipos de variáveis no Titanic

COLUNA	TIPO	POSSÍVEL TRATAMENTO
Age	Numérica	Imputação + escalonamento
Fare	Numérica	Imputação + escalonamento
SibSp, Parch	Numérica (contagem)	Imputação por segurança + escalonamento
Pclass	Numérica com ordem (1 > 2 > 3)	Numérica ou categórica: decisão a justificar
Sex	Categórica	One-Hot Encoding
Embarked	Categórica	Imputação + One-Hot Encoding
Name, Ticket, Cabin, PassengerId	Identificador ou texto livre	Retirar no primeiro modelo
Survived	Alvo	Vira y. Não recebe tratamento de feature.

Por que "male" não entra direto no modelo

Modelos fazem contas: somam, multiplicam, medem distâncias. Não existe conta com a palavra "male".

$$0.8 \times \text{"male"} + 0.3 \times \text{"S"} = ?$$

Os estimadores do scikit-learn que usamos esperam números. Passar texto gera erro ao chamar fit(). Alguns algoritmos e bibliotecas aceitam categorias nativamente, então a necessidade depende do algoritmo, mas no nosso fluxo sempre codificamos.

E POR QUE NÃO TROCAR POR 0, 1, 2?

S → 0

C → 1

Q → 2

O modelo passaria a entender que Q "vale o dobro" de C e que C fica "entre" S e Q. Portos não têm ordem. O One-Hot resolve isso criando uma coluna 0/1 por categoria.

Dados ausentes: o que é NaN e por que não apagar

NaN (Not a Number) é o marcador do pandas para "valor desconhecido". A maioria dos modelos do scikit-learn não aceita NaN no fit().

sem apagar nada

891

linhas

`dropna(subset=["Age"])`

714

linhas · 20% dos passageiros perdidos

`df.dropna()`

183

linhas · 79% perdidos, por causa de Cabin

Apagar também pode distorcer os dados: se as idades faltam mais em certos grupos (por exemplo, na 3ª classe), o dataset que sobra deixa de representar o navio.

Imputação: média, mediana ou moda

Idades:

22

24

26

28

80

Média = 36

Soma dividida pela quantidade. Um único 80 puxa o valor para cima: ninguém do grupo tem perto de 36.

Mediana = 26

O valor do meio depois de ordenar. Resiste a extremos. Por isso é uma boa candidata para Age.

Moda

O valor mais frequente. Funciona com texto. Para Embarked, a moda é "S" (644 de 889).

Isso não é regra universal. É uma decisão que você escreve e justifica no notebook: "usei a mediana porque Age tem assimetria e a mediana é menos sensível a valores extremos".

SimpleImputer, linha por linha

```
from sklearn.impute import SimpleImputer

imp_num = SimpleImputer(strategy="median")
imp_cat = SimpleImputer(strategy="most_frequent")

# só para visualizar; no projeto quem
# chama fit é o Pipeline, e só no treino
imp_num.fit(X_train[["Age"]])
imp_num.statistics_ # mediana aprendida
```

strategy="median"

preenche NaN com a mediana da coluna.

strategy="most_frequent"

preenche com a moda. Serve para texto.

fit(X_train)

aprende o valor de preenchimento. Olha só para o treino.

statistics_

mostra o que foi aprendido. Depois, transform() aplica esse mesmo valor ao teste.

03 · REVISÃO TEÓRICA · PARTE 3

X , y e a separação treino / teste

A Aula 5.2: o conjunto de teste fica trancado até o fim.

Definindo X e y, linha por linha

```
X = df.drop(columns="Survived")  
y = df["Survived"]
```

`df.drop(columns="Survived")` devolve uma cópia do DataFrame sem a coluna alvo. O df original não muda.

X recebe essa cópia: uma tabela com todas as features.

`df["Survived"]` seleciona só a coluna alvo. **y** é uma Series de 0 e 1.

PERGUNTA PARA A TURMA

Survived pode continuar dentro de X?

Não. Seria entregar o gabarito junto com a prova: o modelo "acerta" tudo no treino e não aprende nada útil. Esse é o exemplo mais simples de data leakage.

Treino e teste

O modelo aprende no treino. O teste simula passageiros que o modelo nunca viu.

DATASET COMPLETO · 891 LINHAS

Treino · ~80% · 712

X_train, y_train

Teste · 179

X_test, y_test

X_train

features para aprender

y_train

respostas para aprender

X_test

features para prever no fim

y_test

gabarito para conferir no fim

train_test_split, linha por linha

```
from sklearn.model_selection import (  
    train_test_split)  
  
X_train, X_test, y_train, y_test = (  
    train_test_split(  
        X, y,  
        test_size=0.2,  
        random_state=42,  
        stratify=y,  
    ))
```

X_train, X_test, y_train, y_test

quatro saídas, sempre nesta ordem.

test_size=0.2

20% das linhas vão para o teste.

random_state=42

fixa a semente: o mesmo split toda vez que rodar.
Reprodutibilidade.

stratify=y

mantém a proporção 62% / 38% de Survived em treino e teste.

03 · REVISÃO TEÓRICA · PARTE 4

Data leakage

Quando informação do teste, ou da resposta, chega ao modelo durante o treino.

O que é vazamento de dados

CONCEITO

O modelo usa, no treino, uma informação que não estaria disponível no momento real da previsão.

INTUIÇÃO

É como ver a prova antes de fazê-la. A nota sobe, mas não mede o que você sabe.

CONSEQUÊNCIA

A métrica fica otimista demais e cai quando o modelo encontra dados novos. Foi o "99% que não sobrevive" da Aula 5.2.

Quatro formas comuns de vazamento

1 · Alvo como entrada

Survived dentro de X. O modelo copia a resposta.

2 · Estatística global

Mediana de Age ou média de Fare calculadas com as 891 linhas, incluindo as do teste.

3 · Olhar o teste repetidamente

Ajustar o modelo até o score de teste subir. O teste vira treino disfarçado.

4 · Pré-processar antes do split

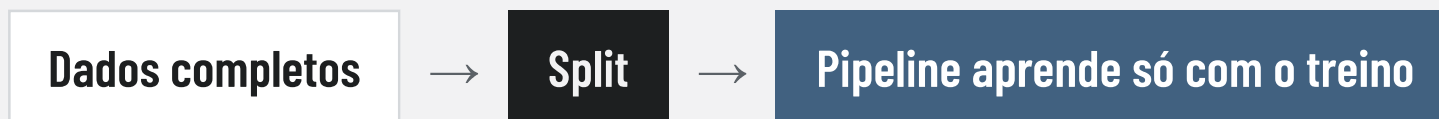
fit_transform no dataset inteiro e só depois separar treino e teste.

A ordem das etapas decide se há vazamento

ERRADO



CORRETO



Regra prática: qualquer passo que calcula algo a partir dos dados (mediana, média, desvio, lista de categorias) só pode fazer fit no treino.

0 motivo: o teste deve continuar desconhecido

SIGA A MEDIANA DE AGE

1. Você calcula a mediana com as 891 linhas.
2. Parte desse número veio dos 179 passageiros do teste.
3. O treino foi preenchido com um valor que "conhece" o teste.
4. Na avaliação, o teste já não é totalmente novo para o processo.

No mundo real, o passageiro novo chega sozinho.

Não dá para recalcular a mediana incluindo ele. O teste só simula bem esse momento se nada dele tiver sido usado antes. No Titanic o efeito numérico da mediana é pequeno; em outros datasets, como vimos na 5.2, pode ser enorme. O hábito é o que importa.

03 · REVISÃO TEÓRICA · PARTE 5

Pré-processamento, ColumnTransformer e Pipeline

A Aula 5.3: tudo empacotado em um único objeto.

Três ferramentas, três problemas

SimpleImputer

Resolve a ausência de dados

Troca NaN por um valor aprendido no treino.

Age: NaN → 28.0

StandardScaler

Coloca números em escala comparável

Subtrai a média e divide pelo desvio padrão do treino.

Fare: 512.33 → 9.7

OneHotEncoder

Transforma categorias em números

Cria uma coluna 0/1 para cada categoria vista no treino.

Sex: male → [0, 1]

Valores aproximados, calculados com as 891 linhas só para ilustrar. No projeto, os valores vêm do treino.

StandardScaler: por que mudar a escala

Age vai de 0 a 80. Fare vai de 0 a 512. Para modelos que comparam ou somam features, o número maior tende a dominar só por ser maior.

$$z = (x - \text{média}) / \text{desvio}$$

Depois do scaler, cada coluna fica com média 0 e desvio 1. Um valor $z = 2$ significa "2 desvios acima da média", seja idade ou tarifa.

ANTES



DEPOIS



Barras representam a amplitude típica de cada coluna.

OneHotEncoder: uma coluna por categoria

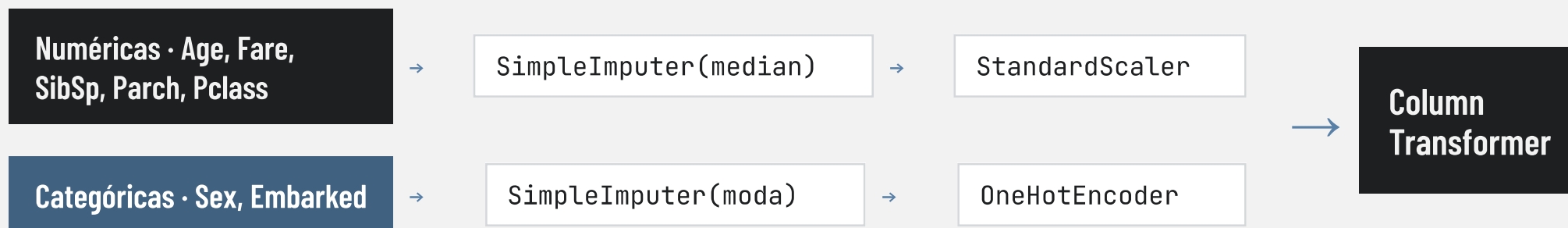
Embarked		Embarked_C	Embarked_Q	Embarked_S
S	→	0	0	1
C		1	0	0
Q		0	1	0
S		0	0	1

Nenhuma categoria "vale mais" que outra: cada uma é só presente (1) ou ausente (0).

`handle_unknown="ignore"` evita erro se o teste trouxer uma categoria que o treino não viu.

ColumnTransformer: um caminho para cada tipo de coluna

Não faz sentido escalonar "male" nem aplicar One-Hot em Fare. Cada tipo segue seu próprio caminho, e no fim tudo volta a ser uma tabela só.



Código, parte 1: quais colunas vão para cada caminho

```
num_cols = ["Age", "Fare", "SibSp",  
            "Parch", "Pclass"]  
cat_cols = ["Sex", "Embarked"]  
  
X = X[num_cols + cat_cols]
```

`num_cols`

lista com os nomes das colunas numéricas.

`cat_cols`

lista com os nomes das categóricas.

`X[num_cols + cat_cols]`

mantém só as colunas escolhidas. Name, Ticket, Cabin e PassengerId saem aqui. Faça isso antes do split.

Pclass entrou como numérica. Colocá-la em `cat_cols` também seria válido, se justificado.

Código, parte 2: os dois caminhos

```
num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
```

`Pipeline([...])`

uma lista de etapas executadas em ordem.

`("imputer", ...)`

cada etapa é uma tupla: um nome que você escolhe e o objeto.

`num_pipe`

primeiro preenche, depois escalona. A ordem importa: o scaler não aceita NaN.

`cat_pipe`

primeiro preenche com a moda, depois codifica.

Código, parte 3: juntando no ColumnTransformer

```
from sklearn.compose import (  
    ColumnTransformer)  
  
preprocessor = ColumnTransformer([  
    ("num", num_pipe, num_cols),  
    ("cat", cat_pipe, cat_cols),  
])
```

`ColumnTransformer([...])`

recebe uma lista de caminhos.

`("num", num_pipe, num_cols)`

nome do caminho, o que aplicar e em quais colunas.

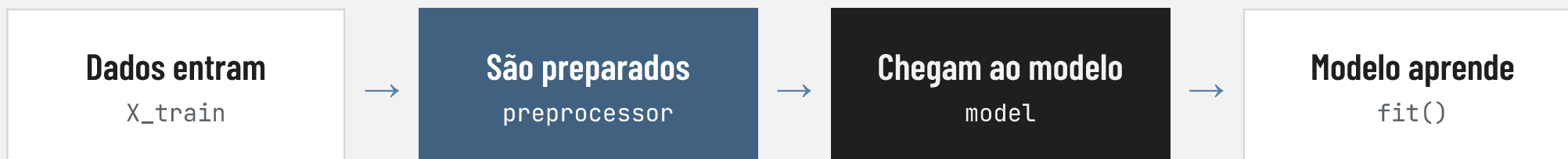
`preprocessor`

devolve uma única matriz numérica: 5 colunas numéricas + as colunas do One-Hot.

Colunas que não estão em nenhuma lista são descartadas (padrão `remainder="drop"`).

Pipeline: uma linha de montagem

Numa cozinha de restaurante, o ingrediente passa sempre pelas mesmas estações, na mesma ordem. Ninguém pula etapa, e o prato sai igual para qualquer cliente.



Na hora de prever, o dado novo passa pelas mesmas estações, usando o que foi aprendido no treino.

Código, parte 4: o Pipeline final

```
modelo = RandomForestClassifier(  
    random_state=42)  
  
pipe = Pipeline([  
    ("preprocessor", preprocessor),  
    ("model", modelo),  
])  
  
pipe.fit(X_train, y_train)  
y_pred = pipe.predict(X_test)
```

`modelo`

um classificador já visto no módulo (a Random Forest apareceu na 5.5). Por enquanto, tratamos como caixa-preta.

`pipe.fit(X_train, y_train)`

aprende imputação, escala, categorias e o modelo, tudo só com o treino.

`pipe.predict(X_test)`

aplica as mesmas transformações ao teste, sem reaprender nada, e devolve 0 ou 1.

Três razões para usar Pipeline

Reduz erros

Um fit e um predict. Não há como esquecer de transformar o teste ou aplicar etapas fora de ordem.

Evita vazamento

O fit acontece só nos dados que você passa. Com `X_train`, nada do teste entra nas estatísticas.

Prepara o laboratório

A validação cruzada da Revisão 2 recebe o pipe inteiro e refaz o pré-processamento dentro de cada fold.

Três perguntas antes de seguir

- 1 Você chamou `imputer.fit_transform(X)` e só depois fez o split. Qual o problema?
- 2 Por que Embarked passa por imputação com moda e não com mediana?
- 3 Um colega obteve 62% de acurácia no Titanic. Isso é um bom resultado?

Respostas

1

Vazamento. A mediana foi calculada com linhas que depois viraram teste. O fit deve acontecer só no treino, de preferência dentro do Pipeline.

2

Embarked é texto (S, C, Q). Não existe mediana de portos. A moda escolhe a categoria mais comum, "S".

3

Não. Responder sempre "não sobreviveu" já dá 61,6%. O modelo praticamente não aprendeu nada além da proporção das classes.

Exercício guiado: nove passos no Titanic

Abra um notebook novo no Colab. Nos próximos slides, complete os espaços ___ e responda às perguntas em uma célula de texto antes de executar.

1 Carregar o dataset

2 Ver as primeiras linhas

3 Verificar dimensões

4 Procurar ausentes

5 Identificar X e y

6 Separar numéricas e categóricas

7 Fazer o split

8 Montar o ColumnTransformer

9 Montar o Pipeline

Passos 1 a 4: carregar e explorar

```
import pandas as pd

# 1. carregar
df = pd.read_csv("___")
# 2. primeiras linhas
df.__()
# 3. dimensões
df.___
# 4. ausentes por coluna
df.___.__()
```

RESPONDA NO NOTEBOOK

Quantas linhas e colunas o dataset tem? O que cada linha representa?

Quais colunas têm ausentes? Qual você trataria e qual retiraria?

Por que Age precisa de tratamento?

Passos 5 e 6: X, y e tipos de coluna

```
# 5. X e y
X = df.drop(columns="___")
y = df["___"]

# 6. tipos de coluna
num_cols = [___]
cat_cols = [___]
X = X[num_cols + cat_cols]
```

RESPONDA NO NOTEBOOK

Por que Survived não aparece em X?

Por que Sex não pode entrar diretamente no modelo?

Pclass ficou em qual lista? Justifique em uma frase.

Passo 7: separar treino e teste

```
X_train, X_test, y_train, y_test = (  
    train_test_split(  
        X, y,  
        test_size=___,  
        random_state=42,  
        stratify=___,  
    ))  
  
print(X_train.shape, X_test.shape)
```

RESPONDA NO NOTEBOOK

Quantas linhas ficaram em cada conjunto?

Compare `y_train.mean()` e `y_test.mean()`. O que o `stratify` garantiu?

Até o fim do notebook, quando você pode olhar `X_test`?

Passos 8 e 9: ColumnTransformer e Pipeline

```
num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="___")),
    ("scaler", ___()),
])
cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="___")),
    ("onehot", ___(handle_unknown="ignore")),
])
preprocessor = ColumnTransformer([
    ("num", num_pipe, ___),
    ("cat", cat_pipe, ___),
])
pipe = Pipeline([("preprocessor", preprocessor),
                  ("model", modelo)])
pipe.fit(___, ___)
```

RESPONDA NO NOTEBOOK

Quem aprende a mediana: treino ou teste?

Por que o imputer vem antes do scaler?

Rode `pipe.predict(X_test[:5])`. Por que não precisou transformar `X_test` antes?

Erros que aparecem no laboratório

ERRO	COMO EVITAR
fit_transform no dataset inteiro antes do split	Split primeiro; o fit fica dentro do Pipeline.
Chamar fit ou fit_transform no X_test	No teste, só predict (ou transform).
Esquecer Survived dentro de X	Confira X.columns logo após o drop.
df.dropna() no dataset inteiro	Imputar; retirar só a coluna Cabin.
Coluna em nenhuma lista, ou em duas	Compare num_cols + cat_cols com X.columns.
Split sem random_state	Fixe a semente para que o resultado seja reproduzível.

O que do notebook do laboratório você já tem

PRONTO HOJE

EDA

Preparação dos dados

Pipeline

Treinamento (pipe.fit)

REVISÃO 2

Validação e validação cruzada

Avaliação e métricas

Interpretação dos resultados

Para a próxima aula: salve o notebook do exercício. Ele será o ponto de partida da Revisão 2.

Mapa mental do Dia 1

1 · Problema

Supervisionado, classificação binária.
Linha = passageiro. y = Survived.

2 · EDA

head, shape, info, describe, isnull, value_counts. Cada resultado vira uma decisão.

3 · X, y e split

drop do alvo, 80/20, random_state, stratify. Teste trancado.

4 · Pré-processamento

Imputer para NaN, Scaler para escala, OneHot para texto.

5 · ColumnTransformer

Caminho numérico e caminho categórico, unidos em uma matriz.

6 · Pipeline

preprocessor + model. Um fit no treino, um predict no teste.

ATRAVESSA TUDO Data leakage: nenhum fit vê o teste.

Documentação oficial

Pipelines e ColumnTransformer	scikit-learn.org/stable/modules/compose.html
Imputação de ausentes	scikit-learn.org/stable/modules/impute.html
Pré-processamento	scikit-learn.org/stable/modules/preprocessing.html
Armadilhas comuns e vazamento	scikit-learn.org/stable/common_pitfalls.html
train_test_split	scikit-learn.org/.../sklearn.model_selection.train_test_split.html
pandas: dados ausentes	pandas.pydata.org/docs/user_guide/missing_data.html